

LAB SESSION 12

MERGE SORT

Introduction

Merge Sort is a popular and efficient sorting algorithm that follows the Divide and Conquer technique. In this algorithm, the given array is divided into two halves until each sub-array contains only one element. Since a single element is always sorted, the algorithm then merges these smaller sorted arrays step by step to produce the final sorted array.

Merge Sort works by repeatedly dividing the problem into smaller sub-problems and then combining the solutions. The merging process compares elements from two sorted arrays and arranges them in the correct order.

One of the main advantages of Merge Sort is that its time complexity remains $O(n \log n)$ in all cases (best, average, and worst). Therefore, it is considered a stable and reliable sorting algorithm.

The objective of this lab experiment is to implement the Merge Sort algorithm, understand its working process, and analyze its performance through practical programming.

Algorithm: Merge Sort

Input:

Array A with n elements

Output:

Sorted array

MERGE SORT(A, left, right)

1. Start
2. If $\text{left} < \text{right}$ then
 - a. Find middle index:
 $\text{mid} = (\text{left} + \text{right}) / 2$
 - b. Call `MERGE_SORT(A, left, mid)`
 - c. Call `MERGE_SORT(A, mid + 1, right)`
 - d. Call `MERGE(A, left, mid, right)`
3. End If
4. Stop

MERGE(A, left, mid, right)

1. Create temporary arrays:

- LeftArray from A[left] to A[mid]
 - RightArray from A[mid+1] to A[right]
- 2. Set pointers:
 i = 0 (for LeftArray)
 j = 0 (for RightArray)
 k = left (for original array)
- 3. While i < size of LeftArray AND j < size of RightArray do
 - a. If LeftArray[i] ≤ RightArray[j] then
 - A[k] = LeftArray[i]
 - i = i + 1
 - b. Else
 - A[k] = RightArray[j]
 - j = j + 1
 - c. k = k + 1
- 4. Copy remaining elements of LeftArray (if any)
- 5. Copy remaining elements of RightArray (if any)
- 6. Stop

Time Complexity

- Best Case: **$O(n \log n)$**
- Average Case: **$O(n \log n)$**
- Worst Case: **$O(n \log n)$**

CODE:

```
#include <iostream>    // Library for input and output

#include <vector>       // Library to use vector (dynamic array)

using namespace std;   // Allows use of standard names without std::

// Function to merge two sorted arrays

vector<double> merge(vector<double> left, vector<double> right) {

    vector<double> result;    // Vector to store merged sorted result

    int i = 0, j = 0;        // Two pointers for left and right arrays

    // Compare elements of both arrays until one array finishes

    while (i < left.size() && j < right.size()) {

        // If element in left array is smaller

        if (left[i] < right[j]) {

            result.push_back(left[i]);    // Add element to result
```

```

        i++;          // Move pointer of left array
    }
    else {
        result.push_back(right[j]); // Add element to result
        j++;          // Move pointer of right array
    }
}

// Add remaining elements of left array (if any)
while (i < left.size()) {
    result.push_back(left[i]);
    i++;
}

// Add remaining elements of right array (if any)
while (j < right.size()) {
    result.push_back(right[j]);
    j++;
}

return result; // Return merged sorted array
}

// Merge Sort function
vector<double> mergeSort(vector<double> arr) {

    // Base case: if array has 1 or 0 elements it is already sorted
    if (arr.size() <= 1) {
        return arr;
    }

    // Find middle index of the array

```

```

int mid = arr.size() / 2;

// Create left half of the array
vector<double> leftHalf(arr.begin(), arr.begin() + mid);

// Create right half of the array
vector<double> rightHalf(arr.begin() + mid, arr.end());

// Recursively sort the left half
vector<double> sortedLeft = mergeSort(leftHalf);

// Recursively sort the right half
vector<double> sortedRight = mergeSort(rightHalf);

// Merge the two sorted halves
return merge(sortedLeft, sortedRight);
}

int main() {
    // Initialize unsorted array
    vector<double> unsortedArr = {3, 7, 6, -10, 15, 23.5, 55, -13};

    // Call mergeSort function
    vector<double> sortedArr = mergeSort(unsortedArr);

    // Print sorted array
    cout << "Sorted array: ";

    for (double num : sortedArr) {
        cout << num << " ";
    }

    return 0; // End of program
}

```

Task 1: Implement Merge Sort

Objective:

Write a program to implement the Merge Sort algorithm to sort an array of integers in ascending order.

Description:

- Create an array of unsorted numbers.
- Apply the Merge Sort algorithm.
- Display the array **before sorting** and **after sorting**.

Example Input:

8, 3, 1, 7, 0, 10, 2

Expected Output:

0, 1, 2, 3, 7, 8, 10

Task 2: Sort User Input Using Merge Sort

Objective:

Write a program that takes input from the user and sorts the elements using **Merge Sort**.

Description:

- Ask the user to enter the number of elements.
- Take array elements as input.
- Use Merge Sort to arrange them in ascending order.
- Display the sorted array.